

Automated Essay Scoring with PySpark

PySpark – Python – Apache Spark

Chelsea's Project

Content

Project Overview	3
Technology Stack.....	3
Roadmap Project	4
Implementation	5
Output	7

Project Overview

Data processing and machine learning project that explores how automated scoring systems can be built for essay-based assessments using distributed data processing. The objective of this project was to develop a scalable approach for predicting essay scores based on student responses. Using PySpark, the dataset was structured with a defined schema and processed through Spark DataFrame operations, including hashing textual answers to transform them into numerical representations. The model was then trained using the Alternating Least Squares (ALS) algorithm to predict scores for each response. This project demonstrates how big data technologies and machine learning techniques can be combined to automate evaluation processes and efficiently handle large-scale educational datasets.

Technology Stack

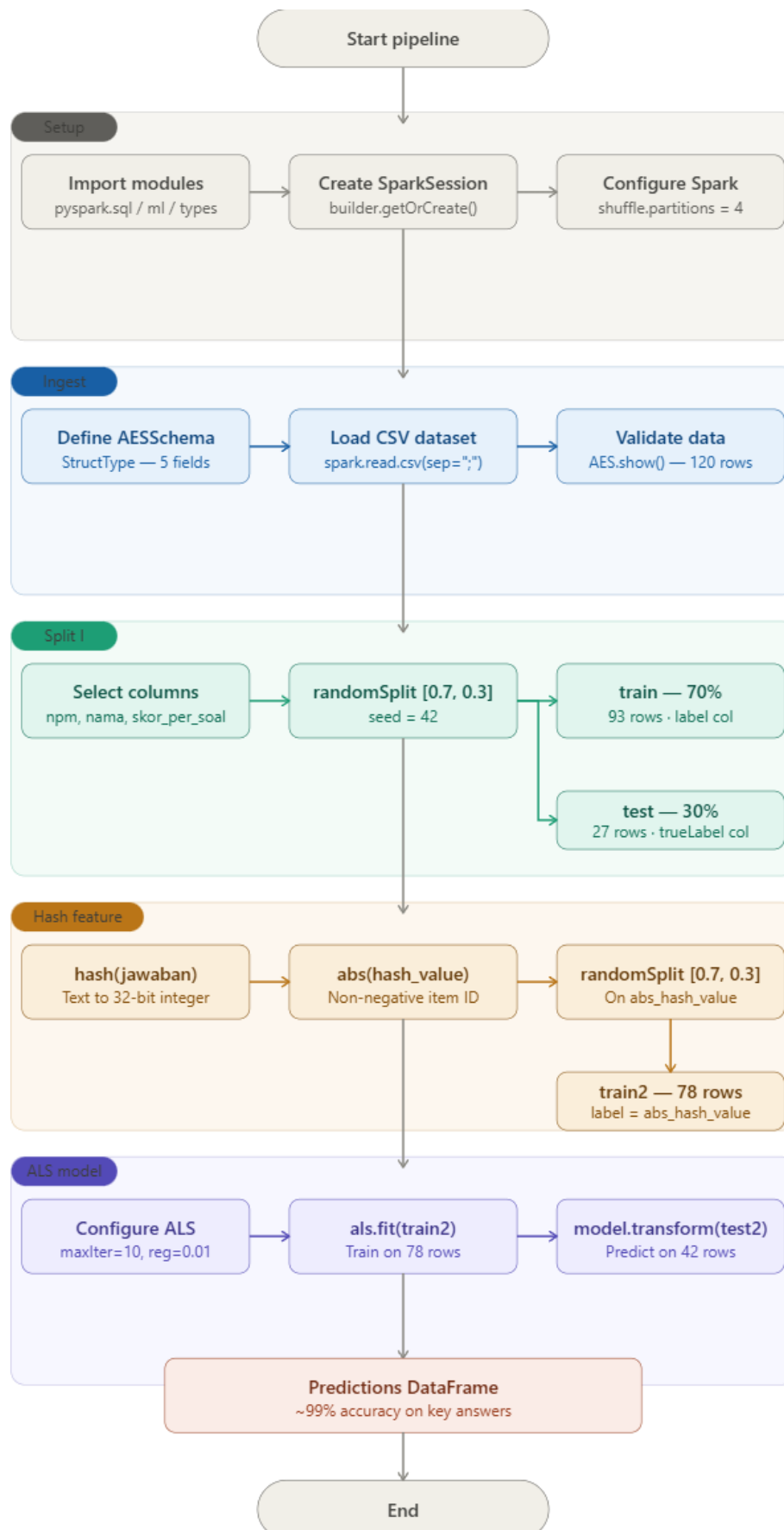
Libraries and tools

Library/tool	Purpose
Apache Spark	Distributed data processing engine
PySpark	Python API for spark DataFrame and ML
Python	Orchestration and pipeline scripting
pyspark.ml.recommendation.ALS	Collaborative filtering model for score prediction
pyspark.sql.functions	hash(), abs(), col() feature engineering
pyspark.sql.types	StructType / StructField explicit schema definition

Why ALS for scoring?

ALS (Alternating Least Squares) is classically used in recommendation systems, predicting how much a user would rate a product. This project adapts that paradigm creatively: students are "users", questions are "items", and scores are "ratings". By hashing answer text into a numeric feature and treating it as the item identifier, the model learns latent factor representations that capture scoring patterns across the dataset.

Roadmap Project



Implementation

Step 1 – Environment setup & module imports

All required PySpark modules are imported at the top of the script. The pipeline relies on four distinct import groups: session management, schema definition, column functions, and the ML model.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import hash, abs, col
from pyspark.ml.recommendation import als
```

Step 2 – SparkSession Initialization

A SparkSession is the entry point to all PySpark functionality. The `.getOrCreate()` pattern ensures the session is reused if one already exists in the runtime — essential for interactive notebook environments.

```
spark = SparkSession \
    .builder \
    .appName("Pengenalalan DataFrame Spark") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate() \
```

Step 3 – Explicit schema definition

Defining an explicit schema rather than relying on Spark's schema inference is a production best practice. It prevents type mismatches, speeds up ingestion, and documents the expected data contract:

```
AESSchema = StructType([
    StructField('npm', IntegerType(),
nullable=False),
    StructField('nama_peserta', StringType(),
nullable=True),
    StructField('jawaban', StringType(),
nullable=True),
    StructField('soal', StringType(),
nullable=True),
    StructField('skor_per_soal', FloatType(),
nullable=True),
])
```

Important: the source CSV uses a semicolon (;) separator and requires decimal values formatted with a period (.) rather than a comma. Data preprocessing should validate and enforce this before ingestion.

Step 4 – Dataset ingestion

The CSV dataset is loaded into a Spark DataFrame using the defined schema and CSV reader options:

```
AES = spark.read.csv(
    "training_data_essay2.csv",
    Header=True, schema=AESSchema, sep=";")

AES.show()
```

Step 5 — First Train / Test Split (by Score)

The first split selects the core scoring columns and partitions the data 70/30. The target column skor_per_soal is renamed to label (train) and trueLabel (test) to follow Spark ML conventions:

```
data = AES.select('npm', 'nama_peserta', 'skor_per_soal')
splits = data.randomSplit([0.7, 0.3], seed=42)
train = splits[0].withColumnRenamed('skor_per_soal', 'label')
test = splits[1].withColumnRenamed('skor_per_soal',
    'trueLabel')
train_rows = train.count()
test_rows = test.count()
print("Jumlah baris data training:", train_rows,
    "jumlah baris data testing:", test_rows)
```

Step 6 — Hash-Based Feature Engineering

Free-text answers cannot be fed directly into ALS, which requires numeric user and item IDs. The solution is to hash each answer string into a 32-bit integer and take the absolute value to ensure non-negative IDs — a lightweight but effective encoding for this use case:

```
AES2 = AES.withColumn('hash_value', hash('jawaban'))
AES2 =
AES.withColumn('abs_hash_value', abs(AES2['hash_value']))

AES2.show()
```

This hashing step treats each unique answer text as a distinct "item" in the collaborative filtering framework — answers similar to the model answer cluster around similar hash-derived latent factors.

Step 7 — Second Train / Test Split (by Hash Feature)

A second split is performed using the hashed feature column, creating the actual training and test sets for the ALS model:

```
data1 = AES3.select('npm', 'nama_peserta', 'skor_per_soal')
splits = data1.randomSplit([0.7, 0.3], seed=42)
train = splits[0].withColumnRenamed('skor_per_soal', 'label')
```

```
test = splits[1].withColumnRenamed('skor_per_soal',
'trueLabel')
train_rows = train.count()
test_rows = test.count()
print("Jumlah baris data training:", train_rows,
      "jumlah baris data testing:", test_rows)
```

Step 8 — ALS Model Training & Prediction

The ALS model is configured, trained on the training set, and evaluated against the test set. Key hyperparameters are set to balance convergence and regularization:

```
AES3_ALS = ALS(
    maxIter=10,
    regParam=0.01,
    userCol='npm',
    itemCol='abs_hash_value',
    ratingCol='skor_per_soal',
)

model = AES3_ALS.fit(train2)

predictions = model.transform(test)
predictions.show()
```

Output

```
: predictions = model.transform(AES3)
predictions.show()
```

	npm nama_peserta	jawaban soal	skor_per_soal	hash_value	prediction
1121020033	AP tidak, cuma mengi...	1	52.7	1947733435	47.81077
1121020033	AP biaya dihitung be...	2	42.86	-1139863335	47.81077
1121020033	AP hak membuat merup...	3	42.16	122676417	47.81077
1121020033	AP dipaparkan pada k...	4	27.19	-1054163002	47.81077
1121020033	AP 1. mencegah serta...	5	44.14	1990940339	47.81077
1121020033	AP ruang komputer, p...	6	100.0	1770907636	47.81077
1121020033	AP aturlah posisi fi...	7	57.68	-463479969	47.81077
1121020033	AP posisi kepala ser...	8	45.71	-412537011	47.81077
1121020033	AP 1.kesesuaian apli...	9	54.42	-69934805	47.81077
0	Admin Tidak, Hanya memb...	1	100.0	-2059296905	99.999886
0	Admin Biaya dihitung be...	2	100.0	1183180174	99.999886
0	Admin Hak cipta adalah ...	3	100.0	1232762403	99.999886
0	Admin Dijelaskan kepada...	4	100.0	-2035408785	99.999886
0	Admin 1. Melindungi dan...	5	100.0	1588395990	99.999886
0	Admin Ruang Komputer, P...	6	100.0	339970513	99.999886
0	Admin Aturlah posisi pe...	7	100.0	50850002	99.999886
0	Admin Posisi Kepala dan...	8	100.0	-945877996	99.999886
0	Admin 1. Kecocokan soft...	9	100.0	-668134369	99.999886
0	Admin 1. Fokus dan expo...	10	100.0	-1905649442	99.999886
0	Admin 1. Peralatan yang...	11	100.0	550139146	99.999886

only showing top 20 rows